

Intuition

Smart Contract Security Assessment

Audit dates: Mar 04 — Mar 09, 2026

Overview

About C4

Code4rena (C4) is a competitive audit platform where security researchers, referred to as Wardens, review, audit, and analyze codebases for security vulnerabilities in exchange for bounties provided by sponsoring projects.

During the audit outlined in this document, C4 conducted an analysis of the Intuition smart contract system. The audit took place from March 04 to March 09, 2026.

Following the C4 Audit, 3 wardens ([Oxastronatey](#), [gesha17](#), and [jerry0422](#)) reviewed the mitigations for all identified issues; the mitigation review report is appended below the audit report.

Final report assembled by Code4rena.

Summary

The C4 analysis yielded an aggregated total of 2 unique vulnerabilities. Of these vulnerabilities, 0 received a risk rating in the category of HIGH severity and 2 received a risk rating in the category of MEDIUM severity.

Additionally, C4 analysis included 107 QA reports compiling issues with a risk rating of LOW severity or informational.

All of the issues presented here are linked back to their original finding, which may include relevant context from the judge and Intuition team.

Scope

The code under review can be found within the [C4 Intuition repository](#), and is composed of 5 smart contracts written in the Solidity programming language and includes 844 lines of Solidity code.

The code in C4's Intuition repository was pulled from:

- Repository: <https://github.com/OxIntuition/intuition-contracts-v2>
- Commit hash: ef69071aaacb7ee6fec91267c9b2904aff0352e6
- Repository: <https://github.com/OxIntuition/intuition-contracts-v2-periphery>
- Commit hash: b026521fe26db8249cd0795b62ec480fd8c848e8

Severity Criteria

C4 assesses the severity of disclosed vulnerabilities based on three primary risk categories: high, medium, and low/informational.

High-level considerations for vulnerabilities span the following key areas when conducting assessments:

- Malicious Input Handling
- Escalation of privileges
- Arithmetic
- Gas use

For more information regarding the severity criteria referenced throughout the submission review process, please refer to the documentation provided on [the C4 website](#), specifically our section on [Severity Categorization](#).

Medium Risk Findings (2)

[\[M-01\] Unsigned validity window metadata](#)

This was found by [V12](#) and selected as the primary submission.

Submitted by [v12](#), also found by [Ox1h3r](#), [Oxbrett8571](#), [Oxgreazy](#), [OxPhantomSec](#), [Oxpiken](#), [OxUnderflow](#), [adriro](#), [akleeko22](#), [Akshay1012](#), [ameng](#), [Antigone4224](#), [archi_upside](#), [assafhh](#), [astaraudit](#), [atilla055](#), [binbin2803](#), [choguun168](#), [cryptoflops](#), [Dinadash](#), [Dinesh11G](#), [dipanshuchhanikar](#), [Hakeem_is_here](#), [happykilling](#), [HawkShadows25](#), [jhongamersx](#), [Josh4324](#), [kimnoic](#), [konstanis](#), [meshaqRapha](#), [MinionTechs](#), [Ni9htNi9ht](#), [Olami978355](#), [Owen_smart](#), [oxp_tr125](#), [qwer](#), [raz-uh](#), [retroactivized](#), [rishiRich](#), [silverologist](#), [valicera](#), [Volleyking](#), [w3bster](#), [WhiteKnightK](#), [XDZIBECX](#), [Yoh](#), [yol](#), and [Zbugs](#)

- `protocol/wallet/AtomWallet.sol` [#L285](#)

Targets

- `_validateSignature` (AtomWallet)

Affected Locations

- `AtomWallet._validateSignature`: Single finding location

Description

The signature validation path extracts `validUntil` and `validAfter` from the trailing 12 bytes of `userOp.signature`, but the recovered signer is computed only over `userOpHash` with an EIP-191 prefix. Because the hash does not include the time-window metadata, those values are not authenticated by the signer. Any relayer can alter the appended validity window while keeping the same 65-byte ECDSA signature, and the recovered address will still match

`owner()`. This defeats the intended time-based restrictions and makes signature validity windows effectively attacker-controlled.

Root cause

The `validUntil/validAfter` metadata is appended to the signature but never incorporated into the hashed message that is signed and recovered, so it is not cryptographically bound to the signer.

Impact

An attacker observing a signed user operation can extend or remove its validity window and resubmit it later, even if the signer intended it to expire quickly or become valid only after a certain time. This enables execution of operations outside the signer's intended timeframe, potentially causing unwanted transfers or actions after the user believed the request had expired.

Proof of Concept

[View Detailed Proof of Concept](#)

Remediation

Status: Complete

Explanation

Bind `validUntil` and `validAfter` to the signed payload by hashing them with `userOpHash` when the signature includes the validity suffix, preventing relayers from altering the window without invalidating the signature.

Patch

```
diff --git a/src/protocol/wallet/AtomWallet.sol
b/src/protocol/wallet/AtomWallet.sol
--- a/src/protocol/wallet/AtomWallet.sol
+++ b/src/protocol/wallet/AtomWallet.sol
@@ -294,7 +294,11 @@
         (uint48 validUntil, uint48 validAfter, bytes memory signature) =

        _extractValidUntilAndValidAfterFromSignature(userOp.signature);

-        bytes32 hash = keccak256(abi.encodePacked("\x19Ethereum Signed
Message:\n32", userOpHash));
+        bytes32 signedHash = userOpHash;
+        if (userOp.signature.length == 77) {
```

```

+         signedHash = keccak256(abi.encodePacked(userOpHash,
validUntil, validAfter));
+     }
+     bytes32 hash = keccak256(abi.encodePacked("\x19Ethereum Signed
Message:\n32", signedHash));

    (address recovered, ECDSA.RecoverError recoverError, bytes32
errorArg) = ECDSA.tryRecover(hash, signature);

```

Intuition mitigated:

- Update AtomWallet signature parsing/validation to treat malformed/invalid signatures as validation failures rather than reverting.
- For 77-byte signatures, sign and verify a digest that commits to (userOpHash, validUntil, validAfter) before applying the EIP-191 prefix.
- Add/adjust unit tests to cover malformed signatures, invalid s values, legacy 77-byte format, and metadata tampering.

Status: Mitigation confirmed. Full details in reports from [gesha17](#), [jerry0422](#), and [Oxastronatey](#).

[M-02] Epoch-boundary checkpoints retroactively qualify for the previous epoch's rewards

Submitted by [Oxastronatey](#), also found by [Ox15](#), [Oxanmol](#), [OxApple](#), [OxDemon](#), [Oxscater](#), [Oxsolisec](#), [aestheticbhai](#), [ahahaHard1k](#), [akhilmanga](#), [Alan_Clan_67](#), [b_void](#), [beer](#), [BlockKnock](#), [dna-thug](#), [Drynooo](#), [gesha17](#), [Hakeem_is_here](#), [happykilling](#), [hiram](#), [hodlturk](#), [jerry0422](#), [jhongamersx](#), [kb520](#), [kindOdev](#), [legat](#), [meganpark](#), [NexusAudits](#), [nstatoshi](#), [oreztker](#), [qed](#), [QuillAudits_Team](#), [saamin](#), [sgtpepper](#), [skipper](#), [solarizer_ai](#), [th3_hybrid](#), [udaykiranpedda](#), [Waiyaki](#), [winfunc](#), [XDZIBECX](#), and [y4y](#)

- /protocol/emissions/CoreEmissionsController.sol [#L178-L180](#)
- /protocol/emissions/CoreEmissionsController.sol [#L207-L213](#)
- /protocol/emissions/TrustBonding.sol [#L194-L213](#)
- /protocol/emissions/TrustBonding.sol [#L475-L492](#)

TrustBonding snapshots veTRUST at epochTimestampEnd(epoch), but currentEpoch() already advances at that exact timestamp. Because VotingEscrow now resolves historical checkpoints with an inclusive $ts \leq t$ lookup, a lock created, topped up, or extended at the first second of epoch N+1 is still counted inside epoch N's reward snapshot, letting an attacker steal or dilute the previous epoch's emissions without actually being bonded during that epoch.

This is distinct from the known post-mortem underflow issue ([POST-MORTEM.md#L22-L57](#)). That outage required a checkpoint strictly after the boundary ($t_s > t$); this issue exploits a checkpoint exactly at the boundary ($t_s == t$), which the fixed binary-search lookup now intentionally includes ([VotingEscrow.sol#L544-L576](#), [VotingEscrow.sol#L590-L623](#)).

Root Cause

1. Controller epoch arithmetic makes `epochTimestampEnd(n)` equal to the first moment of epoch $n+1$

[CoreEmissionsController.sol#L178-L180](#)

```
function _calculateEpochTimestampEnd(uint256 epoch) internal view returns
(uint256) {
    return _START_TIMESTAMP + (epoch * _EPOCH_LENGTH) + _EPOCH_LENGTH;
    // <-- returns the first timestamp of epoch+1
}
```

[CoreEmissionsController.sol#L207-L213](#)

```
function _calculateTotalEpochsToTimestamp(uint256 timestamp) internal
view returns (uint256) {
    if (timestamp < _START_TIMESTAMP) {
        return 0;
    }

    return (timestamp - _START_TIMESTAMP) / _EPOCH_LENGTH;
    // <-- at timestamp == epochTimestampEnd(n), currentEpoch() is
    already n+1
}
```

The controller-side epoch arithmetic makes `epochTimestampEnd(n)` equal to the first moment of epoch $n+1$, not the last in-epoch moment of n .

2. TrustBonding allocates rewards from boundary snapshots, so the inclusive choice directly feeds reward accounting

[TrustBonding.sol#L194-L213](#)

```
function totalBondedBalanceAtEpochEnd(uint256 epoch) public view returns
(uint256) {
    if (epoch > currentEpoch()) {
        revert TrustBonding_InvalidEpoch();
    }
}
```

```

        return _totalSupply(_epochTimestampEnd(epoch));
        // <-- previous-epoch rewards use the inclusive boundary timestamp as
the snapshot time
    }

function userBondedBalanceAtEpochEnd(address account, uint256 epoch)
public view returns (uint256) {
    if (account == address(0)) {
        revert TrustBonding_ZeroAddress();
    }

    if (epoch > currentEpoch()) {
        revert TrustBonding_InvalidEpoch();
    }

    return _balanceOf(account, _epochTimestampEnd(epoch));
    // <-- a checkpoint created exactly at that boundary is eligible for
this snapshot
}

```

[TrustBonding.sol#L475-L492](#)

```

function _userEligibleRewardsForEpoch(address account, uint256 epoch)
internal view returns (uint256) {
    if (account == address(0)) {
        revert TrustBonding_ZeroAddress();
    }

    if (epoch > currentEpoch()) {
        revert TrustBonding_InvalidEpoch();
    }

    uint256 userBalance = userBondedBalanceAtEpochEnd(account, epoch);
    uint256 totalBalance = totalBondedBalanceAtEpochEnd(epoch);
    // <-- reward allocation depends entirely on those boundary snapshots

    if (userBalance == 0 || totalBalance == 0) {
        return 0;
    }

    return userBalance * _emissionsForEpoch(epoch) / totalBalance;
}

```

3. VotingEscrow checkpoint writes and inclusive historical lookups admit boundary-second mutations

[VotingEscrow.sol#L314-L316](#)

```
u_new.ts = block.timestamp;
u_new.blk = block.number;
user_point_history[addr][user_epoch] = u_new;
// <-- lock/create/top-up/extend writes a checkpoint at the exact
transaction timestamp
```

[VotingEscrow.sol#L544-L576](#)

```
if (point_history[_mid].ts <= _ts) {
    _min = _mid;
    // <-- global checkpoint lookup is inclusive: ts == snapshot time is
    selected
} else {
    _max = _mid - 1;
}
```

[VotingEscrow.sol#L590-L623](#)

```
if (user_point_history[addr][_mid].ts <= _ts) {
    _min = _mid;
    // <-- user checkpoint lookup is inclusive too
} else {
    _max = _mid - 1;
}
```

[VotingEscrow.sol#L630-L638](#)

```
uint256 target_epoch = _find_user_timestamp_epoch(addr, _t,
user_point_epoch[addr]);
// <-- a user checkpoint created at _t is treated as part of the balance
snapshot at _t
```

[VotingEscrow.sol#L732-L736](#)

```
uint256 target_epoch = _find_timestamp_epoch(t, epoch);
// <-- same rule for total supply
```


Lock mutations in `VotingEscrow` always checkpoint at `block.timestamp`, and both the global and per-user historical lookups now deliberately select the latest checkpoint whose timestamp is $\leq t$. That means a checkpoint written exactly at `epochTimestampEnd(prevEpoch)` is included in `prevEpoch`'s reward snapshot.

4. `claimRewards` targets the previous epoch whose snapshot still admits brand-new checkpoints

[TrustBonding.sol#L348-L363](#)

```
uint256 currentEpochLocal = currentEpoch();

if (currentEpochLocal == 0) {
    revert TrustBonding_NoClaimingDuringFirstEpoch();
}

uint256 prevEpoch = currentEpochLocal - 1;
uint256 rawUserRewards = _userEligibleRewardsForEpoch(msg.sender,
prevEpoch);
// <-- once the boundary is reached, the contract claims prevEpoch,
//      but prevEpoch's snapshot still includes checkpoints created at
//      that same boundary
```

So once the boundary timestamp is reached, the contract immediately claims the previous epoch, but that previous-epoch snapshot still admits brand-new checkpoints created in the same second.

Internal Pre-conditions

The satellite controller must hold claimable emissions for the target epoch, which is the normal operating condition for `claimRewards()`.

External Pre-conditions

The attacker must be able to submit a lock mutation at the exact timestamp returned by `epochTimestampEnd(prevEpoch)`. An EOA can do this through ordinary `VotingEscrow` entry points such as `create_lock`, `increase_amount`, or `increase_unlock_time`. The exploit does not require any privileged role. EOA access is allowed because the inherited gate only blocks contract callers unless whitelisted.

Attack Path

1. The attacker waits until $t = \text{epochTimestampEnd}(\text{prevEpoch})$.

2. At that exact timestamp, `currentEpoch()` already returns `prevEpoch + 1` because epoch arithmetic is integer-division based on $(\text{timestamp} - \text{start}) / \text{epochLength}$.
3. In that same second, the attacker calls `create_lock`, `increase_amount`, or `increase_unlock_time`. `VotingEscrow` writes a checkpoint with `ts = block.timestamp`, i.e. `ts = epochTimestampEnd(prevEpoch)`.
4. `claimRewards()` now targets `prevEpoch`, and `_userEligibleRewardsForEpoch()` computes balances from `_balanceOf/_totalSupply` at `epochTimestampEnd(prevEpoch)`.
5. Because the historical lookup is inclusive ($ts \leq t$), the attacker's brand-new boundary checkpoint is treated as part of `prevEpoch`'s snapshot. The attacker receives rewards for an epoch they did not actually participate in.
6. If nobody else was bonded during `prevEpoch`, the attacker can capture the entire epoch's emissions by joining only at the first second of the next epoch. If others were bonded, the attacker still retroactively dilutes them.

Impact

This is direct reward theft. The cleanest version is at the epoch-0/epoch-1 boundary: the attacker can join only when epoch 1 has already started, yet still claim epoch 0 emissions. For epochs 0 and 1, the personal-utilization path is automatically 100%, so the exploit does not even need MultiVault activity to get through the utilization layer.

The bug also generalizes beyond the very first epoch. Any existing `veTRUST` holder can retroactively increase their previous-epoch voting weight by topping up or extending at the first second of the next epoch, which steals emissions from honest participants every cycle. Because the attacker's capital is only locked, not spent, the attack cost is mainly timing and temporary capital commitment, while the extracted value is live emissions. I would rate this High.

Negative Check (why blockers don't apply)

- Access control does not stop it. The relevant lock-mutating entry points are inherited `VotingEscrow` user flows, and EOAs can call them permissionlessly.
- `claimRewards()` itself works exactly as intended; the bug is the historical snapshot timestamp it relies on. The claim path simply consumes the corrupted-at-boundary snapshot.
- The known underflow fix does not prevent this. It changed historical lookup behavior to select checkpoints with $ts \leq t$ ([POST-MORTEM.md \(PR #126 context\)](#), [VotingEscrow.sol#L544-L576](#), [VotingEscrow.sol#L590-L623](#)); that inclusive rule is exactly what makes the boundary checkpoint count toward the old epoch.
- This is not the prior zero-amount/utilization-driven repeat-claim issue (Zero-amount claims bypass claimed check) documented in the published V12 findings

([README.md#L30-L34](#), [code_423n4_2026_03_intuition_main_0a19e25_findings.md#L256-L275](#)). This issue does not rely on utilization gaming or missing claims; it is purely a temporal snapshot-admission bug.

Mitigation

Make the epoch-end reward snapshot exclusive.

The simplest fix is to snapshot at `epochTimestampEnd(epoch) - 1` instead of `epochTimestampEnd(epoch)`, so only voting power that existed strictly before the next epoch began is eligible. A more invasive but cleaner alternative is to redefine the controller's "epoch end" helper to be an inclusive in-epoch timestamp and keep that convention consistent across `currentEpoch()`, `epochAtTimestamp()`, `_balanceOf()`, and `_totalSupply()`.

Add regression tests for:

- `create_lock` exactly at `epochTimestampEnd(n)`
- `increase_amount` exactly at `epochTimestampEnd(n)`
- `increase_unlock_time` exactly at `epochTimestampEnd(n)`
- immediate `claimRewards()` for `n`

Proof of Concept

[View Detailed Proof of Concept](#)

Local Verification

Command run locally:

```
forge test --match-path tests/PoCEpochBoundaryRewards.t.sol --match-test
test_boundaryJoinRetroactivelyEarnsPreviousEpochRewards -v
```

Output:

```
Ran 1 test for
tests/PoCEpochBoundaryRewards.t.sol:PoCEpochBoundaryRewards
[PASS] test_boundaryJoinRetroactivelyEarnsPreviousEpochRewards() (gas:
523951)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 8.57ms
(765.96µs CPU time)

Ran 1 test suite in 150.34ms (8.57ms CPU time): 1 tests passed, 0 failed,
0 skipped (1 total tests)
```

[Intuition mitigated:](#)

- Updated CoreEmissionsController epoch-end calculation to be boundary-exclusive across epochs ($\text{end} = \text{start} + (\text{epoch}+1)*\text{length} - 1$).
- Added per-epoch budget guardrails in TrustBonding.claimRewards (revert when exhausted, clamp to remaining budget otherwise).
- Updated and added unit tests to reflect the new closed-interval semantics and to regression-test boundary eligibility and budget invariants.

Status: Mitigation confirmed. Full details in reports from [jerry0422](#), [gesha17](#), and [Oxastronatey](#).

Low Risk and Informational Issues

For this audit, 107 QA reports were submitted by wardens compiling low risk and informational issues. The [QA report highlighted below](#) by [jerry0422](#) received the top score from the judge. 9 Low-severity findings were also submitted individually, and can be viewed [here](#).

The following wardens also submitted QA reports: [Ox23r0](#), [Ox4non](#), [Ox5ul3x](#), [Oxanmol](#), [OxD4n13l](#), [Oxdwrđ](#), [OxFBI](#), [Oxki](#), [OxMaze](#), [OxMehediSec](#), [OxMosh](#), [Oxscater](#), [Oxshdax](#), [AcsonBarbosa](#), [adriro](#), [aestheticbhai](#), [AgengDev](#), [ah_mo](#), [Albary](#), [Alex1984](#), [AriF9212](#), [assafhh](#), [astaraudit](#), [binbin2803](#), [BlackAdam](#), [BlockKnock](#), [Bombz](#), [boolafish](#), [c0pp3rscr3w3r](#), [ChainSentry](#), [chuvak](#), [Cold_Exploit](#), [dadwawd1](#), [dajneem23](#), [Danishniyaz786](#), [Deivitto](#), [Dps4356](#), [Drothon](#), [emerald7017](#), [EZeek33](#), [Fade_cyrpto](#), [Gendolf](#), [gkrastenov](#), [guiz](#), [HalfBloodPrince](#), [hashmate](#), [Homb](#), [I1iveF0rTh1Sh1t](#), [igdbase](#), [InAllHonesty](#), [Invalidatoors](#), [InvisibleKT](#), [jo13](#), [johnyfwesh](#), [K42](#), [kaiserlimp0](#), [Kazopl](#), [keterka](#), [kokman](#), [komane007](#), [konstantinos193](#), [KonstantinVelev](#), [krish_prober](#), [legat](#), [LeoGold](#), [mathiaspel](#), [mbuba666](#), [merlin_san](#), [mistwood](#), [mooneth](#), [Orhukl](#), [Oxhsn](#), [peazzycole](#), [pindarev](#), [prapandey031](#), [Prateekhh](#), [princekay](#), [qed](#), [QuillAudits_Team](#), [reee0001](#), [Riceee](#), [Rikka](#), [SarveshLimaye](#), [Sauron-sol](#), [securehash1](#), [shadow0](#), [shyzo](#), [silverologist](#), [striver_raj](#), [superpraise](#), [SymmaTe](#), [the_haritz](#), [TheCarrot](#), [Tiro](#), [udaykiranpedda](#), [unbiasedcodex](#), [UzeyirCh](#), [v12](#), [victorrrr](#), [Vinay](#), [vip3r](#), [vt729830](#), [Wsecure](#), [yonko](#), [zenoh](#), and [Zislasher](#).

[01] quoteExactInput silently swallows quoter reverts and returns 0, collapsing slippage protection in downstream swaps

- [intuition-contracts-v2-periphery/contracts/TrustSwapAndBridgeRouter.sol#L215-L223](#)

quoteExactInput wraps the Slipstream quoter call in a bare try/catch and returns amountOut = 0 for every failure condition:

```
function quoteExactInput(bytes calldata path, uint256 amountIn) external
returns (uint256 amountOut) {
    try ICLQuoter(slipstreamQuoter).quoteExactInput(path, amountIn)
returns (
    uint256 quotedAmountOut, uint160[] memory, uint32[] memory,
uint256
    ) {
        amountOut = quotedAmountOut;
    } catch {
        amountOut = 0; // @audit swallows all errors
    }
}
```

This makes it impossible for a caller to distinguish between a legitimate zero-output quote and a failed/reverted quoter call. The function is designed to be used as a preflight check — a frontend or integration contract calls it, applies a slippage tolerance, and passes the result as minTrustOut to swapAndBridgeWithETH / swapAndBridgeWithERC20.

When the quoter reverts (transient RPC issue, stale oracle, deprecated pool), the following chain of events silently removes all slippage protection:

1. Frontend calls quoteExactInput → quoter reverts internally → function returns 0
2. Frontend computes $\text{minTrustOut} = 0 * (1 - \text{slippage}\%) = 0$
3. Frontend calls swapAndBridgeWithETH(path, minTrustOut=0, recipient)
4. Swap executes with amountOutMinimum = 0 — any output amount is accepted
5. A sandwich bot sandwiches the transaction, extracting maximum value; the user receives near-zero TRUST

The attack only requires a transient quoter failure — a realistic condition on any live AMM — combined with standard integrator behavior of deriving minTrustOut from the preflight quote.

Recommended mitigation steps

Remove the catch block and let the revert propagate naturally. Callers that need graceful degradation can handle it themselves:

```
function quoteExactInput(bytes calldata path, uint256 amountIn) external
returns (uint256 amountOut) {
    (amountOut,,, ) = ICLQuoter(slipstreamQuoter).quoteExactInput(path,
amountIn);
}
```

Alternatively, return a success flag so callers can distinguish error from a valid zero:

```

function quoteExactInput(bytes calldata path, uint256 amountIn)
    external
    returns (uint256 amountOut, bool success)
{
    try ICLQuoter(slipstreamQuoter).quoteExactInput(path, amountIn)
returns (
    uint256 quotedAmountOut, uint160[] memory, uint32[] memory,
uint256
    ) {
        amountOut = quotedAmountOut;
        success = true;
    } catch {
        amountOut = 0;
        success = false;
    }
}

```

[02] `block.timestamp` used as swap deadline renders expiry protection ineffective in `TrustSwapAndBridgeRouter`

- intuition-contracts-v2-periphery/contracts/TrustSwapAndBridgeRouter.sol [#L110](#)
- intuition-contracts-v2-periphery/contracts/TrustSwapAndBridgeRouter.sol [#L164](#)

Both `swapAndBridgeWithETH` and `swapAndBridgeWithERC20` pass deadline: `block.timestamp` to Slipstream's `exactInput`:

```

// L107-L114 (swapAndBridgeWithETH)
ISlipstreamSwapRouter.ExactInputParams({
    path: path,
    recipient: address(this),
    deadline: block.timestamp, // @audit always passes
    amountIn: swapEth,
    amountOutMinimum: minTrustOut
})

```

```

// L161-L168 (swapAndBridgeWithERC20)
ISlipstreamSwapRouter.ExactInputParams({
    path: path,
    recipient: address(this),
    deadline: block.timestamp, // @audit always passes

```

```

    amountIn: amountIn,
    amountOutMinimum: minTrustOut
  })

```

The Slipstream router enforces `require(block.timestamp <= deadline)`. Because `deadline` is set to `block.timestamp` at execution time, this check is a tautology and always passes. A transaction that sits in the mempool for an extended period — due to low gas pricing, network congestion, or sequencer behavior — will execute at an arbitrarily late time with no expiry.

The `minTrustOut` parameter provides slippage protection (output floor) but is orthogonal to deadline protection. It does not prevent execution of a transaction the user intended to expire: if market prices shift adversely but remain within the user's slippage tolerance, the transaction executes at a worse rate than the user expected at submission time, with no recourse.

Recommended mitigation steps

Accept a caller-supplied `deadline` parameter in both functions and forward it to `ExactInputParams`:

```

function swapAndBridgeWithETH(
    bytes calldata path,
    uint256 minTrustOut,
    address recipient,
    uint256 deadline          // added
) external payable ...

```

This allows users to set a meaningful expiry at submission time, consistent with standard Uniswap V3 integration patterns.

[03] `_epochsPerYear()` integer truncation causes systematic APY underestimation in `getSystemApy()` and `getUserApy()`

`src/protocol/emissions/TrustBonding.sol` [#L442-L444](#)

`_epochsPerYear()` computes the number of epochs per year using integer division:

```

function _epochsPerYear() internal view returns (uint256) {
    return YEAR /
    ICoreEmissionsController(satelliteEmissionsController).getEpochLength();
}

```

Integer division truncates the remainder. For the protocol's default 14-day epoch (epochLength = 1,209,600):

```
YEAR / epochLength = 31,536,000 / 1,209,600 = 26 (exact: 26.071...)
```

Both `getSystemApy()` and `getUserApy()` multiply per-epoch rewards by this truncated value, causing all returned APY figures to be systematically lower than the true value.

Impact

Purely a display inaccuracy. The actual per-epoch reward distribution is unaffected. Frontend UIs and integrators consuming `getSystemApy()`/`getUserApy()` will show APY values ~0.27% lower than reality for 14-day epochs, which may influence user decisions.

Recommended mitigation steps

Move the division to the end of each multiplication to minimise precision loss, rather than scaling `_epochsPerYear()` itself (which would require cascading changes to all callers):

```
// getSystemApy() - before
uint256 emissionsPerYear = _emissionsForEpoch(_currEpoch) *
    _epochsPerYear();

// getSystemApy() - after
uint256 emissionsPerYear = _emissionsForEpoch(_currEpoch) * YEAR
    /
    ICoreEmissionsController(satelliteEmissionsController).getEpochLength();
```

Apply the same pattern in `getUserApy()`. This eliminates the intermediate truncation with no downstream interface changes.

[04] `TrustBonding.pause()` only blocks `claimRewards()`, leaving all inherited `VotingEscrow` state-changing functions callable while paused

- `protocol/emissions/TrustBonding.sol` [#L348](#)
- `external/curve/VotingEscrow.sol` [#L400](#)
- `external/curve/VotingEscrow.sol` [#L406](#)
- `external/curve/VotingEscrow.sol` [#L422](#)
- `external/curve/VotingEscrow.sol` [#L485](#)

`TrustBonding` inherits `VotingEscrow` and adds `PausableUpgradeable` to support an emergency stop mechanism. However, when `NotPaused` is applied only to `claimRewards()`:


```

34
8
34 function claimRewards(address recipient) external whenNotPaused
9 nonReentrant {

```

The four inherited VotingEscrow entry points that mutate escrow state carry no pause check:

FUNCTION	LOCATION
create_lock()	VotingEscrow.sol#L400
increase_amount()	VotingEscrow.sol#L406
increase_unlock_time()	VotingEscrow.sol#L422
withdraw()	VotingEscrow.sol#L485

None of these are overridden in TrustBonding to add whenNotPaused.

As a result, triggering pause() does not halt the escrow state machine. During an emergency pause:

- Users can create new veTRUST positions (create_lock)
- Users can increase the size of existing positions (increase_amount)
- Users can extend lock durations (increase_unlock_time)
- Users with expired locks can withdraw principal (withdraw)

All of these alter supply, the per-user locked balances, and the global checkpoint history that drives _totalSupply() and _balanceOf() — the two functions that determine pro-rata reward allocation at epoch boundaries. If an epoch boundary falls during a pause window, the final bonded-balance snapshots used for reward distribution upon unpaue will reflect position changes that occurred while the operator believed the system was frozen.

The root cause is in TrustBonding (in-scope), not in VotingEscrow (out-of-scope): the in-scope contract inherits mutating functions and fails to override them with the pause guard it introduces.

Recommended mitigation steps

Override each inherited mutator in TrustBonding and add whenNotPaused:

```

function create_lock(uint256 _value, uint256 _unlock_time)
    external override nonReentrant onlyUserOrWhitelist notUnlocked
whenNotPaused
{

```

```

        _create_lock(_value, _unlock_time);
    }

    function increase_amount(uint256 _value)
        external override nonReentrant onlyUserOrWhitelist notUnlocked
        whenNotPaused
    {
        _increase_amount(_value);
    }

    function increase_unlock_time(uint256 _unlock_time)
        external override nonReentrant onlyUserOrWhitelist notUnlocked
        whenNotPaused
    {
        _increase_unlock_time(_unlock_time);
    }

    function withdraw()
        external override nonReentrant whenNotPaused
    {
        _withdraw();
    }

```

If allowing withdrawals during a pause is intentional (to let users safely exit during an incident), `withdraw()` may be excluded from the above, but this should be an explicit design decision documented in the code.

[05] `deposit_for` pulls tokens from the lock holder instead of the caller, allowing any third party to forcibly lock a victim's tokens

- `external/curve/VotingEscrow.sol` [#L354-L356](#)
- `external/curve/VotingEscrow.sol` [#L372-L379](#)

`VotingEscrow._deposit_for` always transfers tokens from `_addr` (the lock holder), regardless of who is calling:

```

// VotingEscrow.sol L354-356
if (_value != 0) {
    IERC20(token).safeTransferFrom(_addr, address(this), _value);
}

```

`deposit_for` is permissionless — any caller can invoke it for any `_addr` with an active lock:

```
// VotingEscrow.sol L372-379
function deposit_for(address _addr, uint256 _value) external nonReentrant
notUnlocked {
    LockedBalance memory _locked = locked[_addr];
    require(_value > 0);
    require(_locked.amount > 0, "No existing lock found");
    require(_locked.end > block.timestamp, "Cannot add to expired lock.
Withdraw");
    _deposit_for(_addr, _value, 0, _locked,
DepositType.DEPOSIT_FOR_TYPE);
}
```

A user who has given a broad or residual ERC-20 approval to TrustBonding (which inherits VotingEscrow) can have an arbitrary amount of tokens forcibly pulled from their wallet into their own time-lock — with no consent and no recourse until the lock expires.

Attack scenario

1. Alice approves TrustBonding with type(uint256).max (common DeFi UX pattern) and creates a 2-year lock with 10,000 TRUST.
2. An attacker calls `deposit_for(alice, 50_000e18)`.
3. 50,000 TRUST is pulled from Alice's wallet and added to her lock — locked for up to 2 years.
4. The attacker spends zero tokens. Alice has no way to recover her funds until the lock matures.

Recommended mitigation steps

Override `deposit_for` in TrustBonding (or modify `_deposit_for`) to transfer tokens from `msg.sender` rather than `_addr` when `deposit_type == DEPOSIT_FOR_TYPE`. This matches the semantic intent of the function — a third party contributing their own tokens to top up someone else's lock — and eliminates the griefing vector entirely:

```
function deposit_for(address _addr, uint256 _value) external nonReentrant
notUnlocked {
    LockedBalance memory _locked = locked[_addr];
    require(_value > 0);
    require(_locked.amount > 0, "No existing lock found");
    require(_locked.end > block.timestamp, "Cannot add to expired lock.
Withdraw");

    // Transfer from caller, not from the lock holder
    IERC20(token).safeTransferFrom(msg.sender, address(this), _value);
}
```

```

    // Pass _value = 0 to suppress the internal transfer in _deposit_for
    _deposit_for(_addr, _value, 0, _locked,
DepositType.DEPOSIT_FOR_TYPE);
}

```

[06] `_refundExcess()` hard-reverts when caller cannot accept ETH, permanently blocking non-payable contract callers

- `/intuition-contracts-v2-periphery/contracts/TrustSwapAndBridgeRouter.sol`
[#L291-L296](#)
- `/intuition-contracts-v2-periphery/contracts/TrustSwapAndBridgeRouter.sol`
[#L172-L173](#)
- `/intuition-contracts-v2-periphery/contracts/TrustSwapAndBridgeRouter.sol`
[#L198-L199](#)

`_refundExcess()` forwards leftover ETH back to `msg.sender` via a low-level call and unconditionally reverts on failure:

```

29
6
29 function _refundExcess(uint256 refundAmount) internal {
7
29     if (refundAmount > 0) {
8
29         (bool success,) = msg.sender.call{ value: refundAmount }("");
9
30         if (!success) revert
0 TrustSwapAndBridgeRouter_ETHRefundFailed();
30     }
1
30 }
2

```

This is invoked in both `swapAndBridgeWithERC20` (L172-173) and `bridgeTrust` (L198-199) whenever `msg.value > bridgeFee`. Any contract caller that lacks a payable `receive()`/`fallback()` — such as a smart account, a multisig, or a protocol integration — will have every call revert the moment a refund is triggered.

The critical aggravating factor is that `bridgeFee` is not statically knowable. It is fetched at execution time via `metaERC20Hub.quoteTransferRemote()` (L152, L192). A contract caller cannot guarantee `msg.value == bridgeFee` exactly; any safe buffer causes `refundAmount >`

0, which causes the revert. This makes the DoS unavoidable by design for a whole class of callers.

Impact

`swapAndBridgeWithERC20` and `bridgeTrust` are permanently unusable for any contract integration that does not accept raw ETH transfers, with no workaround available to the caller short of modifying their own contract.

Recommended mitigation steps

Allow callers to specify a separate `refundRecipient` address (e.g. an EOA they control) so the refund is never sent back to the calling contract:

```
function bridgeTrust(
    uint256 trustAmount,
    address recipient,
    address refundRecipient // new parameter
) external payable ...

// in _refundExcess:
(bool success,) = refundRecipient.call{ value: refundAmount }("");
```

Alternatively, adopt a pull-based pattern: accumulate refunds in a per-sender mapping and expose a `withdrawRefund()` function, eliminating all push-ETH failure modes entirely.

[07] Epoch-end ve balance snapshot enables emission sniping, allowing late lockers to capture disproportionate reward shares

- `protocol/emissions/TrustBonding.sol` [#L475-L492](#)
- `protocol/emissions/TrustBonding.sol` [#L194-L200](#)
- `protocol/emissions/TrustBonding.sol` [#L203-L213](#)

`_userEligibleRewardsForEpoch` allocates epoch N emissions using a single point-in-time ve balance snapshot at `_epochTimestampEnd(N)`:

```
uint256 userBalance = userBondedBalanceAtEpochEnd(account, epoch); //
snapshot at epoch end
uint256 totalBalance = totalBondedBalanceAtEpochEnd(epoch); //
snapshot at epoch end
return userBalance * _emissionsForEpoch(epoch) / totalBalance;
```

Because `_balanceOf(account, t)` (inherited from `VotingEscrow`) is a pure point-in-time interpolation, a lock created one second before `_epochTimestampEnd(N)` produces the same — or higher — `ve` balance at the snapshot than an identical lock held for the full epoch. A 2-year lock minted at epoch-end has zero decay, while the same 2-year lock minted at epoch-start has already decayed by one epoch length, giving the late entrant a strictly larger snapshot balance.

Additionally, `_getPersonalUtilizationRatio` returns `BASIS_POINTS_DIVISOR` (100%) for first-time participants whose epoch `N-1` eligibility was zero (L528-529), so no prior vault activity is required to claim the full pro-rata share on the first claim.

Attack scenario

1. Near the end of epoch `N`, attacker creates a large lock with maximum duration (2 years).
2. `userBondedBalanceAtEpochEnd(attacker, N)` and `totalBondedBalanceAtEpochEnd(N)` are sampled only at the boundary — the attacker's fresh lock has not decayed at all.
3. In epoch `N+1`, attacker calls `claimRewards` and receives a pro-rata share of epoch `N` emissions as if they had provided `ve` weight for the entire epoch; the personal utilization ratio is 100% due to the first-claim exemption.

Recommended mitigation steps

Replace the epoch-end point-in-time snapshot with a time-weighted average balance over the epoch for both `userBondedBalanceAtEpochEnd` and `totalBondedBalanceAtEpochEnd`. If a full TWAB is too gas-intensive, a simpler alternative is to snapshot eligibility at epoch start (i.e., use `_epochTimestampEnd(epoch - 1)` as the reference point), so that locks created or increased during epoch `N` only become reward-eligible from epoch `N+1` onward.

[08] ProgressiveCurve: `maxShares()` and `maxAssets()` are not jointly reachable due to rounding direction mismatch

- `protocol/curves/ProgressiveCurve.sol` [#L65-L69](#)
- `protocol/curves/ProgressiveCurve.sol` [#L125-L135](#)

`ProgressiveCurve` publishes two capacity limits via `maxShares()` and `maxAssets()`. Integrators are expected to treat these as the jointly valid operational ceiling of the curve. However, the two values are computed with opposite rounding conventions, making the advertised (`maxShares`, `maxAssets`) pair impossible to reach via the mint path.

Root cause — `initialize()` computes `MAX_ASSETS` with round-down arithmetic:

```
// ProgressiveCurve.sol L65-69
UD60x18 maxSharesUD = sqrt(wrap(uMAX_UD60x18 / uUNIT));
```

```
UD60x18 maxAssetsUD = mul(PCMath.square(maxSharesUD), HALF_SLOPE);
//                      ^^^^^^^^^^^^^^^^^ round-down  ^^ round-down
MAX_SHARES = unwrap(maxSharesUD);
MAX_ASSETS = unwrap(maxAssetsUD); // floor(floor(maxShares2 / 1e18) *
halfSlope / 1e18)
```

Root cause — `previewMint()` computes `assetsOut` with round-up arithmetic:

```
// ProgressiveCurve.sol L131-132
UD60x18 area      = sub(PCMath.squareUp(sNext), PCMath.square(s));
//                      ^^^^^^^^^ round-up
UD60x18 assetsUD = PCMath.mulUp(area, HALF_SLOPE);
//                      ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ round-up
// assetsOut = ceil(ceil(maxShares2 / 1e18) * halfSlope / 1e18)
```

Because $\text{ceil}(x) \geq \text{floor}(x)$, we have `assetsOut` $\geq$ `MAX_ASSETS` by construction, and `assetsOut` $>$ `MAX_ASSETS` whenever either product is not exactly divisible by `1e18`. The subsequent check in `_checkMintOut` then reverts:

```
// BaseCurve.sol L196-197
function _checkMintOut(uint256 assetsOut, uint256 totalAssets, uint256
maxAssetsCap) internal pure {
    if (assetsOut > maxAssetsCap - totalAssets) revert
    BaseCurve_AssetsOverflowMax();
}
```

An integrator that reads `maxShares()` / `maxAssets()` as hard operational bounds and attempts a mint at the published ceiling will receive an unexpected revert even though the share-side bound check passes cleanly.

Recommended mitigation steps

Compute `MAX_ASSETS` using the same conservative (round-up) arithmetic used by the mint path, so the published value is always a safe upper bound on what `previewMint` will require:

```
UD60x18 maxAssetsUD = PCMath.mulUp(PCMath.squareUp(maxSharesUD),
HALF_SLOPE);
MAX_ASSETS = unwrap(maxAssetsUD);
```

This ensures `assetsOut` $\leq$ `MAX_ASSETS` whenever `shares` + `totalShares` $\leq$ `MAX_SHARES`, making the two published limits jointly and consistently reachable.

[O9] OffsetProgressiveCurve: MAX_ASSETS computed with inverted rounding vs. previewMint, making the published share cap unreachable

- `protocol/curves/OffsetProgressiveCurve.sol` [#L71 — MAX_ASSETS initialization \(round-down\)](#)
- `protocol/curves/OffsetProgressiveCurve.sol` [#L136-137 — previewMint area + multiply \(round-up\)](#)

`OffsetProgressiveCurve.initialize()` computes `MAX_ASSETS` — the advertised maximum asset cost for the full share range — using conservative, rounded-down arithmetic:

```
7
1
7 UD60x18 maxAssetsUD = mul(
2
7     sub(PCMath.square(add(maxSharesUD, OFFSET)),
3 PCMath.squareUp(OFFSET)),
7     HALF_SLOPE
4
7 );
5
```

`previewMint()` computes the asset cost for the same range using the opposite rounding direction at every step:

```
137
138 UD60x18 area = sub(PCMath.squareUp(sNext), PCMath.square(s));
139 UD60x18 assetsUD = PCMath.mulUp(area, HALF_SLOPE);
```

For a mint from `totalShares = 0` to `totalShares = MAX_SHARES` (i.e. `s = OFFSET`, `sNext = MAX_SHARES + OFFSET`).

Every rounding direction is inverted between the two paths:

INITIALIZE	PREVIEWMINT
round down	round up
round up → smaller result	round down → larger result
round down	round up

The combined effect is `assetsOut > MAX_ASSETS`. The subsequent `_checkMintOut` guard:


```

19
7
19 if (assetsOut > maxAssetsCap - totalAssets) revert
8   BaseCurve_AssetsOverflowMax();

```

reverts even though $\text{shares} \leq \text{MAX_SHARES}$ passes the preceding `_checkMintBounds` check. The published `(maxShares, maxAssets)` pair is therefore not jointly reachable: an integrator relying on `maxShares()` and `maxAssets()` as operational limits will encounter an unexpected revert at the boundary.

Recommended mitigation steps

Compute `MAX_ASSETS` in `initialize()` using the same rounding direction as `previewMint()` — i.e. round up throughout — so the stored cap is always $\geq$ the maximum value the mint path can return:

```

UD60x18 maxAssetsUD = PCMath.mulUp(
    sub(PCMath.squareUp(add(maxSharesUD, OFFSET)),
PCMath.square(OFFSET)),
    HALF_SLOPE
);

```

Alternatively, lower `MAX_ASSETS` to the largest value confirmed reachable via `previewDeposit` (which rounds down), though this approach requires more careful analysis of the deposit path's cumulative rounding behaviour.

Detailed Proofs of Concept for the above-listed Low-severity issues may be viewed [here](#).

Mitigation Review

Introduction

Following the C4 audit, 3 wardens, ([Oxastronatey](#), [gesha17](#), and [jerry0422](#)), reviewed the mitigations for all identified issues. Additional details can be found within the [C4 Intuition Mitigation Review repository](#).

Mitigation Review Scope & Summary

During the mitigation review, the wardens confirmed that all in-scope findings were mitigated.

The table below provides details regarding the status of each in-scope vulnerability from the original audit.

ORIGINAL ISSUE	STATUS	MITIGATION URL
S-112	● Mitigation Confirmed	PR 144
S-149	● Mitigation Confirmed	PR 143
S-595	● Mitigation Confirmed	PR 144
S-145 (Low)	● Mitigation Confirmed	PR 143
S-324 (low)	● Mitigation Confirmed	PR 10

Disclosures

C4 audits incentivize the discovery of exploits, vulnerabilities, and bugs in smart contracts. Security researchers are rewarded at an increasing rate for finding higher-risk issues. Audit submissions are judged by a knowledgeable security researcher and disclosed to sponsoring developers. C4 does not conduct formal verification regarding the provided code but instead provides final verification.

C4 does not provide any guarantee or warranty regarding the security of this project. All smart contract software should be used at the sole risk and responsibility of users.